

Mathematical Analysis of Linear Constraint Systems using QR Decomposition

Based on `analyze_system_with_qr` function

1 The Problem: Analyzing $Ax = b$

We are given a system of m linear constraints (equations) in n variables (unknowns), represented in matrix form as:

$$Ax = b$$

where A is an $m \times n$ matrix, x is an $n \times 1$ vector of variables, and b is an $m \times 1$ vector of constants.

Our goal is to analyze this system to determine its properties:

- **Consistent vs. Inconsistent:** Does a solution x exist that satisfies all equations? If not, the system is inconsistent or "conflicting."
- **Under-constrained vs. Over-constrained:** If a solution exists, is it unique, or are there infinitely many solutions (under-constrained)? An over-constrained system has more independent equations than variables.

The Python function `analyze_system_with_qr` uses QR decomposition with column pivoting, a robust numerical method perfect for this task.

2 QR Decomposition with Column Pivoting

Standard QR decomposition factors A as $A = QR$, where Q is an $m \times m$ orthogonal matrix ($Q^T Q = I$) and R is an $m \times n$ upper trapezoidal matrix.

The function `sparseqr.qr` performs a more powerful version: **QR decomposition with column pivoting**. This decomposition is represented as:

$$AP = QR$$

or equivalently, $A = QRP^T$.

- P is an $n \times n$ **permutation matrix** that reorders the columns of A . In your code, this is represented by the vector E , which holds the new order of columns (e.g., $E = [2, 0, 1]$ means column 2 of A comes first, then 0, then 1).
- Q is an $m \times m$ orthogonal matrix.
- R is an $m \times n$ upper trapezoidal matrix.

The pivoting is crucial: it reorders the columns of A such that the linearly independent columns are moved to the "front." This process also reveals the **rank** r of the matrix A (where $r \leq \min(m, n)$). The matrix R has a specific structure based on this rank:

$$R = \begin{pmatrix} R_{11} & R_{12} \\ 0 & 0 \end{pmatrix}$$

Here, R_{11} is an $r \times r$ upper triangular, non-singular (invertible) matrix. The remaining $m - r$ rows of R are all zeros.

3 Transforming the System

We use this decomposition to transform the original problem $Ax = b$ into an equivalent, but much simpler, one.

1. **Substitute the decomposition:**

$$(QRP^T)x = b$$

2. **Multiply by Q^T :** Since Q is orthogonal, $Q^TQ = I$. We multiply both sides on the left by Q^T :

$$(Q^TQ)(RP^T)x = Q^Tb$$

$$RP^T x = Q^T b$$

3. **Introduce new variables y :** Let's define a new vector of permuted variables $y = P^T x$. This y is just x with its elements reordered according to the permutation P .

4. **Define the transformed vector c :** Let $c = Q^T b$. This is what your code calculates as `c_vec = Q.transpose() @ b_col`.

Our original, complex system $Ax = b$ is now transformed into the equivalent, upper-triangular system:

$$Ry = c$$

4 Solving and Analyzing the Transformed System

Now we break this new system down using the block structure of R . We split y and c at row r (the rank):

$$y = \begin{pmatrix} y_1 \\ y_2 \end{pmatrix} \quad \text{and} \quad c = \begin{pmatrix} c_1 \\ c_2 \end{pmatrix}$$

where y_1 and c_1 are $r \times 1$ vectors, and y_2 and c_2 are $(n - r) \times 1$ and $(m - r) \times 1$ vectors, respectively.

Substituting this into $Ry = c$:

$$\begin{pmatrix} R_{11} & R_{12} \\ 0 & 0 \end{pmatrix} \begin{pmatrix} y_1 \\ y_2 \end{pmatrix} = \begin{pmatrix} c_1 \\ c_2 \end{pmatrix}$$

This matrix equation separates into two distinct systems:

$$R_{11}y_1 + R_{12}y_2 = c_1 \quad (\text{System 1}) \tag{1}$$

$$0 \cdot y_1 + 0 \cdot y_2 = c_2 \quad (\text{System 2}) \tag{2}$$

We can now analyze the original system just by looking at these two simple equations.

4.1 Finding Inconsistency (Conflicts)

Look at System 2:

$$0 = c_2$$

This is a set of $m - r$ equations. If *any* element in the vector c_2 is non-zero, this equation is a contradiction (e.g., $0 = 5$). This means no solution y (and thus no solution x) can possibly exist. The system is **inconsistent**.

The vector c_2 is precisely what your code calculates: `c_bottom = c_vec[rank:]`. The **conflict norm**, `conflict_norm = norm(c_bottom)`, is the Euclidean norm ($\|c_2\|_2$) of this "error" vector.

- If `conflict_norm` is very close to zero (e.g., $< 10^{-9}$), the system is **consistent**.
- If `conflict_norm` is significantly greater than zero, the system is **inconsistent**. The magnitude of the norm measures "how inconsistent" the system is.

This c_2 vector represents the part of b that lies outside the column space of A —the part that Ax can never "reach," no matter what x we choose.

4.2 Finding Under-constrained Systems

The system is under-constrained if it has infinitely many solutions. This happens when there are fewer independent constraints (r) than variables (n).

- If $r < n$, the system is **under-constrained**. This is checked by `is_unconstrained = rank < n`.
- Why? If $r < n$, the vector y_2 (which has $n - r$ elements) exists. These are **free variables**. We can choose any values we want for y_2 , and then use System 1 to solve for y_1 :

$$y_1 = R_{11}^{-1}(c_1 - R_{12}y_2)$$

Since R_{11} is invertible, a unique y_1 exists for every choice of y_2 . Since there are infinite choices for y_2 , there are infinite solutions.

4.3 Summary of Conditions

- **Inconsistent (Conflicting):** $\|c_2\| > \epsilon$ (i.e., `conflict_norm > 1e-9`).
- **Consistent, Unique Solution:** $\|c_2\| \leq \epsilon$ and $r = n$.
- **Consistent, Infinite Solutions:** $\|c_2\| \leq \epsilon$ and $r < n$.

5 Finding the "Best" Solution and Residuals

Even if a system is inconsistent, we can find a "best-fit" solution. This is the **least-squares solution**, x_{hat} , which minimizes the error $\|Ax - b\|^2$. The QR decomposition method naturally finds this.

1. **Find a particular solution y_p :** The code finds the simplest solution by setting the free variables y_2 to zero. System 1 becomes:

$$R_{11}y_1 = c_1$$

This is what `y1 = solve_triangular(R11, c1, lower=False)` does. It solves for y_1 . So,

$$y_p = \begin{pmatrix} y_1 \\ 0 \end{pmatrix}.$$

2. **Transform back to x_{hat} :** We convert y_p back to the original variable ordering.

$$x_{\text{hat}} = Py_p$$

This is what the code does with `x_hat[E[:rank]] = y1`. It creates the full-length x_{hat} vector and places the y_1 solutions into the "permuted" slots defined by the first r elements of E .

3. **Calculate the Residual:** The code then computes the **residual vector**, $r_{\text{res}} = b - Ax_{\text{hat}}$.

4. **Identifying Conflicting Constraints:** This residual vector is the key. Each element i of r_{res} shows how "unhappy" or "violated" the i -th constraint is by the best-fit solution x_{hat} .

If the system was perfectly consistent, r_{res} would be a zero vector.

If the system is *inconsistent*, r_{res} will be non-zero. The constraints that are "causing" the conflict will be poorly satisfied by x_{hat} (which tries to satisfy the "majority" of consistent constraints). Therefore, these conflicting constraints will have the largest absolute values in the residual vector.

By sorting the indices of the residual vector by their absolute value, `sorted_indices = np.argsort(np.abs(r_res))` you are creating a ranked list of the constraints, from **most likely to be inconsistent** (largest residual) to least likely.